BASH SCRIPTING FUNDAMENTALS

Projects

Build real automation scripts end to end

Merit Advisory
Bash Scripting Fundamentals Bootcamp
Beginner-friendly lecture materials
July 2026

WHAT WE WILL COVER

Agenda

1. **Section 1: Project habits before coding** — 4 slides
2. **Section 2: Project A sysinfo report basics** — 4 slides
3. **Section 3: Project A sysinfo report details** — 4 slides
4. **Section 4: Project B backup-dir basics** — 4 slides
5. **Section 5: Project B backup-dir polish** — 4 slides
6. **Section 6: Project C log scanner basics** — 4 slides
7. **Section 7: Project C log scanner details** — 4 slides
8. **Section 8: Project D toolbelt CLI basics** — 4 slides

9 **Section 9: Project D toolbelt polish** — 4 slides

10 **Section 10: Documentation and safety** — 4 slides

11 **Section 11: Build plan and acceptance** — 4 slides

12 **Section 12: Common bugs and portfolio polish** — 4 slides

1 Section 1

Project habits before coding

1.1 Start with one sentence requirements · Name scripts after actions

- A project begins with what the script must do
- Keep the first version small enough to finish
- Write requirements before writing commands
- Clear names make command history readable
- Use lowercase words separated by hyphens
- The **name** should describe the main job

Examples

```
# Requirement
echo 'Create a sysinfo report file'
# Scope line
echo 'Input: none, output: report.txt'
# Project names
sysinfo-report
backup-dir
log-scan
```

1.2 Create a small project folder · Use a Bash shebang

- Keep scripts, tests, and samples together
- A predictable layout helps demos
- Avoid mixing generated output with source files
- The shebang tells the OS how to run the script
- env finds bash through PATH
- Keep it as the first line

Examples

```
# Make folders
mkdir -p project/bin project/tests
# Samples folder
mkdir -p project/samples
# Shebang
#!/usr/bin/env bash
# Run file
./bin/sysinfo-report
```

1.3 Use strict mode in projects · Make scripts executable

- Project scripts should fail predictably
- Strict mode catches missing variables and failed commands
- Add specific exceptions where expected
- chmod allows direct script execution
- Version control can remember executable bits
- Test both direct and bash invocation

Examples

```
# Strict header
set -euo pipefail
# Expected no match
grep -q ERROR log || true
# Add execute bit
chmod +x bin/sysinfo-report
# Run directly
./bin/sysinfo-report
```

1.4 Commit after each working slice

- Small commits capture progress safely
- Each slice should run before committing
- A project history is part of your portfolio

Examples

```
# Check status
git status --short
# Commit slice
git add . && git commit -m 'add sysinfo header'
```

2 Section 2

Project A sysinfo report basics

2.1 Sysinfo report requirements · Choose the report file

- The script writes a readable machine report
- It should include host, kernel, uptime, memory, and disk
- The output file path should be predictable
- A variable makes the output path easy to change
- Quote the report path on every use
- Start with one default file

Examples

```
# Requirement note
echo 'sysinfo-report writes report.txt'
# Output name
report=system-report.txt
# Default report
report="system-report.txt"
# Create empty report
: > "$report"
```

2.2 Write a report title · Add a timestamp

- A title makes the report self-explanatory
- Redirect once to create the file
- Append later sections with double greater-than
- Reports should say when they were produced
- Use one consistent date format
- **Capture** command output with substitution

Examples

```
# Title line
echo 'System Information Report' > "$report"
# Separator
echo '========================' >> "$report"
# Timestamp value
now=$(date '+%Y-%m-%d %H:%M:%S')
# Write timestamp
echo "Generated: $now" >> "$report"
```

2.3 Collect the hostname · Collect kernel information

- hostname identifies the machine
- **Store** command output before writing
- Use labels in the report
- uname shows operating system and kernel details
- uname -a is verbose and useful for diagnostics
- Keep the raw line for beginner projects

Examples

```
# Hostname
host=$(hostname)
# Report hostname
echo "Hostname: $host" >> "$report"
# Kernel value
kernel=$(uname -a)
# Report kernel
echo "Kernel: $kernel" >> "$report"
```

2.4 Collect uptime

- uptime shows load and running time
- The **output** is already human-readable
- Append it under a labeled section

Examples

```
# Uptime value
up=$(uptime)
# Report uptime
echo "Uptime: $up" >> "$report"
```

3 Section 3

Project A sysinfo report details

3.1 Add disk usage with df · Add memory with free

- df -h reports human-readable disk usage
- The root filesystem is a useful first target
- **Append** command output directly for tables
- free -h reports memory in readable units
- Not every minimal system has free installed
- A dependency check can explain missing tools

Examples

```
# Disk heading
echo 'Disk usage:' >> "$report"
# Disk command
df -h / >> "$report"
# Memory heading
echo 'Memory:' >> "$report"
# Memory command
free -h >> "$report"
```

3.2 Format sections with blank lines · Group report writing

- Blank lines make command output easier to scan
- Use a tiny helper when repeated
- **Formatting** is part of script quality
- A brace **group** can share one redirection
- This reduces repeated append operators
- Keep the **group** readable

Examples

```
# Blank line
echo >> "$report"
# Section helper
section() { echo; echo "$1"; } >> "$report"
# Grouped output
{ echo 'Report'; date; } > "$report"
# Append group
{ echo; df -h /; } >> "$report"
```

3.3 Print the report path · Test report creation

- A successful script should tell the user what changed
- Write status messages to stderr
- Keep report contents on disk
- The first **test** is that the file exists
- The second **test** is that the file is not empty
- Run tests after each new section

Examples

```
# Done message
echo "wrote $report" >&2
# Show path
printf 'Report: %s
' "$report" >&2
# Exists test
[ -f system-report.txt ] || exit 1
[ -s system-report.txt ] || exit 1
```

3.4 Test report contents

- Search for labels that should always appear
- Avoid testing machine-specific exact values
- Content tests make refactors safer

Examples

```
# Check hostname label
grep -q '^Hostname:' system-report.txt
# Check disk label
grep -q '^Disk usage:' system-report.txt
```

4 Section 4

Project B backup-dir basics

4.1 Backup project requirements · Require one directory argument

- The script archives one directory
- Backups go into a backups folder
- The archive name includes a timestamp
- The input directory is required
- Argument validation happens before work
- Bad usage should exit 2

Examples

```
# Requirement note
echo 'backup-dir DIR creates backups/*.tar.gz'
# Default folder
backup_root=backups
# Arg count
[ "$#" -eq 1 ] || exit 2
# Store arg
src=$1
```

4.2 Validate the source directory · Create the backups directory

- The backup target must be an existing directory
- Use -d for directory tests
- Name the invalid path in the error
- mkdir -p is safe when the directory already exists
- Keep generated archives out of the source tree
- Quote the backup directory path

Examples

```
# Directory check
[ -d "$src" ] || exit 1
# Error message
echo "not a directory: $src" >&2
# Make output dir
mkdir -p "$backup_root"
# Writable output
[ -w "$backup_root" ] || exit 1
```

4.3 Use timestamped archive names · Use basename for readable names

- Timestamps prevent overwriting older backups
- **Use** sortable formats for filenames
- Avoid spaces in generated archive names
- basename removes parent directories
- The archive name can include the source folder name
- Quote the source path passed to basename

Examples

```
# Timestamp
stamp=$(date '+%Y%m%d-%H%M%S')
# Archive path
archive="backups/home-$stamp.tar.gz"
# Base name
name=$(basename "$src")
# Archive name
archive="backups/${name}-${stamp}.tar.gz"
```

4.4 Create tar.gz archives

- tar czf creates compressed gzip archives
- The archive path comes before the source path
- Quote both paths

Examples

```
# Create archive
tar czf "$archive" "$src"
# Verbose archive
tar czvf "$archive" "$src"
```

5 Section 5

Project B backup-dir polish

5.1 Print archive size · Add dry-run mode

- Users want confirmation after backups
- du -h gives a readable size
- **Print** status to stderr
- Dry-run mode shows what would happen
- **It** is important for scripts that create large files
- Implement it before adding destructive cleanup

Examples

```
# Archive size
du -h "$archive"
# Size message
echo "size: $(du -h "$archive" | cut -f1)" >&2
# Dry flag
dry_run=1
# Dry message
[ "$dry_run" = 1 ] && echo "would tar $src"
```

5.2 Route actions through run · Keep the five newest backups

- A run helper centralizes dry-run behavior
- It also makes verbose logging easier
- Call run for commands that change files
- Retention prevents backup folders from growing forever
- Start with a simple **keep** count
- Test retention on copied sample files

Examples

```
# Run helper
run() { echo "+ $*" >&2; "$@"; }
# Use run
run tar czf "$archive" "$src"
# Keep count
keep=5
# List newest
ls -1t backups/*.tar.gz | head -n "$keep"
```

5.3 Delete older backups carefully · Handle tar failures

- Only **delete** files matching this script's pattern
- Review the list before wiring rm
- Use dry-run for retention first
- A failed archive should stop the script
- The error should name the source directory
- Do not run retention after a failed backup

Examples

```
# Old list
ls -1t backups/*.tar.gz | tail -n +6
# Delete old
ls -1t backups/*.tar.gz | tail -n +6 | xargs -r rm
# Tar with die
tar czf "$archive" "$src" || exit 1
# Specific error
echo "backup failed: $src" >&2
```

5.4 Test restore manually

- A **backup** is only useful if it restores
- Extract into a temporary directory
- Check that expected files appear

Examples

```
# Extract test
tar xzf "$archive" -C /tmp/restore-test
# List restore
find /tmp/restore-test -maxdepth 2 -type f
```

6 Section 6

Project C log scanner basics

6.1 Log scanner requirements · Create sample.log

- The script reads one **log** file
- It counts total lines and severity levels
- It prints a short report
- A sample file makes development repeatable
- Include INFO, WARN, and ERROR lines
- Keep the first sample small

Examples

```
# Requirement note
echo 'log-scan FILE prints counts'
# Input variable
log_file=$1
# Sample line
echo 'INFO started' > sample.log
# More sample
echo 'ERROR failed' >> sample.log
```

6.2 Validate the log file · Count total lines

- The scanner needs a readable file
- Use -f before counting
- Exit 2 for missing argument and 1 for missing file
- wc -l gives the total number of lines
- Redirect input to avoid printing the filename
- Store the **count** for report formatting

Examples

```
# Arg count
[ "$#" -eq 1 ] || exit 2
# File check
[ -f "$log_file" ] || exit 1
# Line count
total=$(wc -l < "$log_file")
# Print total
echo "Total lines: $total"
```

6.3 Count ERROR lines · Count WARN lines

- grep -c counts matching lines
- ERROR **count** is usually the headline metric
- Handle zero matches under strict mode
- WARN lines often indicate risk without failure
- Use the same pattern as ERROR counts
- Consistent variables make the report easy

Examples

```
# Error count
errors=$(grep -c 'ERROR' "$log_file" || true)
# Print errors
echo "ERROR: $errors"
# Warn count
warns=$(grep -c 'WARN' "$log_file" || true)
# Print warns
echo "WARN: $warns"
```

6.4 Count INFO lines

- INFO counts help compare normal activity
- The scanner should report all required severities
- Use uppercase patterns for the first version

Examples

```
# Info count
infos=$(grep -c 'INFO' "$log_file" || true)
# Print infos
echo "INFO: $infos"
```

7 Section 7

Project C log scanner details

7.1 Understand grep -c caveats · Show the last errors

- grep returns 1 when there are no matches
- **That** is not a scanner crash
- Use an intentional fallback for counts
- Recent errors are more useful than every error
- Use tail after grep
- Handle the no-error case gracefully

Examples

```
# No match status
grep -c ERROR clean.log
echo $?
# Safe count
count=$(grep -c ERROR clean.log || true)
# Last errors
grep 'ERROR' "$log_file" | tail -n 5
grep 'ERROR' "$log_file" | tail -n 5 || true
```

7.2 Use pipefail with last errors · Format a scanner report

- pipefail may make no matches fail the pipeline
- Decide whether no errors is acceptable
- Add fallback only around that one pipeline
- **Use** labels that are easy to scan
- Group counts before sample lines
- Keep output stable for tests

Examples

```
# Pipeline
set -o pipefail
grep ERROR "$log_file" | tail -n 5
# Expected none
grep ERROR "$log_file" | tail -n 5 || true
# Report header
echo 'Log Scan Report'
printf 'ERROR: %s
' "$errors"
```

7.3 Add a quiet machine-readable mode · Test log counts

- Some callers want just the numbers
- A mode flag can switch output style
- Start with one simple output format
- A sample log gives exact expected counts
- Tests should fail when a count changes
- **Use** command substitution for captured output

Examples

```
# CSV line
printf '%s,%s,%s
' "$errors" "$warns" "$infos"
# Mode variable
format=csv
# Run scanner
output=$(./log-scan sample.log)
grep -q 'ERROR: 1' <<< "$output"
```

7.4 Test no-error logs

- No-error input is a key edge case
- The script should still exit successfully
- The report should show zero errors

Examples

```
# Clean sample
echo 'INFO ok' > clean.log
# Clean test
./log-scan clean.log | grep -q 'ERROR: 0'
```

8 Section 8

Project D toolbelt CLI basics

8.1 Toolbelt project requirements · Read the subcommand

- A **toolbelt** script groups small utilities
- Subcommands choose the action
- The first version can wrap earlier projects
- The first argument selects behavior
- Use a default so missing commands are safe
- Shift after reading the subcommand

Examples

```
# Requirement note
echo 'toolbelt SUBCOMMAND [ARGS]'
# Example commands
toolbelt sysinfo
toolbelt log-scan sample.log
# Read command
cmd="${1:-}"
[ "$#" -gt 0 ] && shift
```

8.2 Write a top-level help message · Use functions for subcommands

- Help should list available subcommands
- Print help for no arguments
- A CLI without help feels unfinished
- Each subcommand belongs in one function
- Functions keep the case router short
- Arguments after shift are passed to the function

Examples

```
# Help function
usage() { echo 'usage: toolbelt COMMAND'; }
# No command
[ -n "$cmd" ] || { usage; exit 2; }
# Function
cmd_sysinfo() { ./sysinfo-report; }
# With args
cmd_backup() { ./backup-dir "$@"; }
```

8.3 Route with case · Return subcommand statuses

- **case** is the standard Bash router for commands
- Each pattern calls one function
- The default case handles unknown commands
- The toolbelt should fail when the subcommand fails
- Do not hide the status with a final echo
- Use **return** in functions and exit at top level

Examples

```
# Case route
case "$cmd" in
  sysinfo) cmd_sysinfo "$@";;
# ...
# Default route
*) usage; exit 2;;
# Return status
cmd_backup() { ./backup-dir "$@"; }
cmd_backup "$@"
exit $?
```

8.4 Use exit 2 for unknown commands

- **Unknown** command is a usage problem
- Show help so the user can recover
- Keep runtime failures as exit 1

Examples

```
# Unknown command
echo "unknown command: $cmd" >&2
# Usage exit
usage
exit 2
```

9 Section 9

Project D toolbelt polish

9.1 Pass remaining arguments safely · Reuse shared helper functions

- $@ preserves argument boundaries when quoted
- Subcommands should receive their own inputs
- Never **pass** unquoted $* in project scripts
- usage, die, and need can be shared across commands
- Keep helpers near the top of the file
- Do not duplicate dependency checks everywhere

Examples

```
# Safe pass
cmd_log_scan "$@"
# Function wrapper
cmd_log_scan() { ./log-scan "$@"; }
# Die helper
die() { echo "$*" >&2; exit 1; }
# Need helper
need() { command -v "$1" >/dev/null || exit 127; }
```

9.2 Add per-command help · Package scripts in bin

- Subcommands often need their own usage text
- Support help before running real work
- This keeps the top-level help short
- A bin directory is familiar for command-line tools
- Keep executable entry points there
- Use README examples that call bin scripts

Examples

```
# Help branch
[ "${1:-}" = --help ] && usage_backup
# Backup help
usage_backup() { echo 'usage: toolbelt backup DIR'; }
# Bin layout
mkdir -p bin
# Move script
mv toolbelt bin/toolbelt
```

9.3 Install locally through PATH · Keep paths relative to the script

- PATH lets users run the tool from any directory
- A local bin directory avoids system installs
- Document the PATH update
- The caller may run the tool from any directory
- Find the **script** directory with BASH_SOURCE
- Use that base path for sibling scripts

Examples

```
# Make local bin
mkdir -p "$HOME/bin"
# Copy tool
cp bin/toolbelt "$HOME/bin/toolbelt"
# Script dir
script_dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
# Sibling script
"$script_dir/backup-dir" "$@"
```

9.4 Run demo commands

- Demo commands prove the CLI is understandable
- Use examples that a reviewer can copy
- Include one success and one help command

Examples

```
# Help demo
bin/toolbelt --help
# Project demo
bin/toolbelt sysinfo
```

10

Section 10

Documentation and safety

10.1 Write a README for scripts · Document dependencies

- A README explains what the project does
- Include requirements, usage, and examples
- Keep it current as behavior changes
- Projects often rely on external commands
- List them so users can prepare
- Check them in the script too

Examples

```
# README title
echo '# Bash Toolbelt' > README.md
# Usage line
echo 'Usage: bin/toolbelt COMMAND' >> README.md
# README dependency
echo 'Requires: bash tar grep' >> README.md
# Runtime check
command -v tar >/dev/null || exit 127
```

10.2 Document exit codes · Use relative path safety

- Exit codes help automation callers
- Explain success, usage errors, and runtime failures
- Keep codes consistent across projects
- Scripts should not depend on the caller's directory
- Resolve paths before using sibling files
- Quote resolved paths

Examples

```
# Exit docs
echo '0 success, 1 failure, 2 usage' >> README.md
# Usage exit
exit 2
# Project root
root=$(cd "$(dirname "$0")/.." && pwd)
# Sample path
sample="$root/samples/sample.log"
```

10.3 Prefer BASH_SOURCE for sourced paths · Keep generated files out of source

- BASH_SOURCE points at the current script file
- $0 can point at the parent command when sourced
- Use BASH_SOURCE in reusable Bash projects
- Generated reports and archives can clutter commits
- Use output directories and ignore patterns
- Review git status before committing

Examples

```
# Dirname pattern
dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
# Project root
root=$(cd "$dir/.." && pwd)
# Ignore reports
echo '*.tar.gz' >> .gitignore
# Check status
git status --short
```

10.4 Show safe destructive commands

- Any rm command needs extra care
- Print what will be removed before deleting
- Pair deletion with dry-run mode

Examples

```
# Preview delete
printf 'delete %s
' "$old_file"
# Guard delete
[ "$dry_run" = 1 ] || rm -- "$old_file"
```

11

Section 11

Build plan and acceptance

11.1 Build one vertical slice · Add one feature at a time

- A vertical slice runs end to end with minimal features
- It proves the script shape early
- **Add** detail after the first successful run
- Small changes make bugs easier to find
- Run the script after each feature
- Avoid rewriting the whole script at once

Examples

```
# Tiny slice
echo 'System Report' > report.txt
# Run slice
./sysinfo-report
# Add hostname
echo "Hostname: $(hostname)" >> report.txt
# Run again
./sysinfo-report && grep Hostname report.txt
```

11.2 Keep an acceptance checklist · Create a manual test checklist

- Acceptance checks define done
- They should be observable from the command line
- Use them before demo day
- Manual checks are useful for beginner projects
- Include success and failure cases
- Record commands so another person can repeat them

Examples

```
# Checklist item
echo '[ ] report file is created'
# Check item
[ -s system-report.txt ] && echo pass
# Success test
./backup-dir samples
# Failure test
./backup-dir missing || echo expected
```

11.3 Automate the easiest checks · Run syntax checks for every script

- Turn repeated manual checks into scripts
- Start with file existence and output labels
- Automated checks build confidence quickly
- Syntax checks are fast and low effort
- They catch broken edits before demos
- Add ShellCheck when available

Examples

```
# Test script
bash tests/sysinfo_test.sh
# Simple assert
grep -q Hostname system-report.txt
# Bash syntax
bash -n bin/toolbelt
# All bin scripts
for f in bin/*; do bash -n "$f"; done
```

11.4 Prepare demo runs

- A demo script removes typing pressure
- Use fresh sample inputs
- Show the output files after commands run

Examples

```
# Demo script
echo './bin/toolbelt sysinfo' > demo.sh
# Run demo
bash demo.sh
```

12

Section 12

Common bugs and portfolio polish

12.1 Common bug: unquoted project paths · Common bug: missing execute bit

- Project demos often use simple paths first
- Real users have spaces in directories
- Test with a path containing a space
- Permission errors confuse first-time users
- Check executable mode before demos
- Document chmod in setup steps

Examples

```
# Space path
mkdir -p 'sample dir'
# Quoted run
./backup-dir 'sample dir'
# Check mode
test -x bin/toolbelt
# Fix mode
chmod +x bin/toolbelt
```

12.2 Common bug: outputs overwritten · Stretch goal: JSON-like output

- Reports and backups can overwrite earlier runs
- Use timestamps when history matters
- Use a fixed name only when replacement is intended
- Machine-readable output makes scripts easier to integrate
- Keep the first version simple
- Escape data carefully in real JSON projects

Examples

```
# Timestamp file
file="report-$(date +%Y%m%d).txt"
# Fixed latest
latest=system-report.txt
# Simple JSON line
printf '{"errors":%s}
' "$errors"
printf 'ERROR: %s
' "$errors"
```

12.3 Stretch goal: configuration file · Portfolio tip: include screenshots or transcripts

- A config file avoids long command lines
- Source only trusted config files
- Validate config values after loading
- A transcript shows the tool working
- Keep sample output short and readable
- Explain what problem each script solves

Examples

```
# Source config
. ./toolbelt.conf
# Default value
keep="${keep:-5}"
# Save transcript
script -q demo.txt ./demo.sh
# Show output
sed -n '1,20p' demo.txt
```

12.4 Final project walkthrough

- Walk through requirements, code, tests, and demo
- Explain one bug you fixed
- End with a clear next improvement

Examples

```
# Walkthrough order
echo 'requirements code tests demo'
# Next step
echo 'Next: add automated retention tests'
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
08 · Projects